moon-slice 项目申报书

1 项目名称

moon-slice — MoonBit 程序切片与依赖图工作台

2 项目简介

moon-slice 是面向 MoonBit 的程序切片与依赖分析工作台。输入一个 `.mbt` 文件（或整个包）与目标语句，它基于 MoonBit 官方 parser 构建控制流图、定义-使用链、控制依赖图与程序依赖图，输出向后切片（这个值从哪来）、向前切片（改这里会影响谁）、死代码与无人调用的私有函数、调用点查询，并给出 JSON / DOT / SARIF / 自包含 HTML 报告与浏览器 playground。

项目面向在 MoonBit 中做重构、调试、代码审查与 CI 质量门禁的库作者和团队：把"这次改动影响了什么""哪段代码已经死了"变成可复现的命令行输出和可上传的 SARIF。已发布到 mooncakes.io（https://mooncakes.io/docs/NaIWaniubi/moon-slice），`moon add` 即可安装；`11 月新增整包扫描按 mtime 增量（217 文件包热扫描 0.2 s）与 unused` 线性化。

3 项目方向与通用性

- 方向：开发者工具 —— LSP、CLI 与开发体验。
- 通用性：不绑定业务域，任意 MoonBit 包都是输入；只依赖官方 parser 的 AST，不要求被分析代码使用特定框架；输出为标准 JSON、SARIF 2.1.0、DOT 与自包含 HTML，可直接接 CI、代码扫描与文档；分析全在本地完成，无网络依赖。

4 预期使用场景

1. **重构前评估影响面**：`review --diff` 对 git diff 的每个新增行做向前切片，输出「改了 1 行 → 影响 2 行」及受影响函数，替代人工通读。
2. **CI 质量门禁**：`dead --dir` / `unused --dir` 配 `--fail-on-findings` 与基线抑制，结果以 SARIF 上传 GitHub code scanning，PR 上直接看到新增死代码。
3. **编辑器内边写边查**：`moon-slice-lsp` 发布死存储 / 不可达 / 整包 unused 诊断，hover 同时给出向后切片与向前影响，quick fix 插入 `// moon-slice: ignore[...]`。
4. **读陌生代码**：选中一行问「这个值从哪来」，切片列出构成它的语句行号，比全文搜索可靠。

5 拟实现的核心功能

- 语句级 IR 与 CFG、reaching definitions、控制依赖（结构化 + 后支配）、PDG；
- 向后 / 向前切片，输出节点集合与源码行号，一行多语句用 `--node` 精确选择；
- 包级过程间切片：跨文件展开被调函数（裸调用整函数体、返回值调用只展开返回 cone、`p.f()` 展开到 `T::f`）；
- 死代码 `dead`（无使用赋值 + 不可达）与 `unused`（整包无人调用的私有函数与固有方法）；
- 就地抑制、SARIF 2.1.0、基线、门禁、GitHub composite action；
- LSP（诊断 / 双向 hover / quick fix）与浏览器 playground；
- 质量证据：381 个测试、585 文件 / 98,384 行真实代码 0 误报、2158 文件上游存活分析 10 处真实死代码。

6 原创性

原创项目，未移植任何现有代码。算法参考 Weiser《Program Slicing》(1981) 与 Ferrante–Ottenstein–Warren《The Program Dependence Graph and Its Use in Optimization》(1987) 的公开论文，实现为独立代码。

7 移植或参考说明

不适用（非移植项目）。依赖 `moonbitlang/parser` 0.4.6 与 `moonbitlang/x` 0.5.1，均为 Apache-2.0；本项目 Apache-2.0，第三方清单见 `THIRD_PARTY.md`。

8 GitHub 仓库链接

https://github.com/NaIWaniubi/moon-slice —— 60+ commits（无空提交与拆分提交），CI 与 Pages 均为绿色，最新发布 `NaIWaniubi/moon-slice@0.10.2`。